

The Virtual Agile Enterprise: Making the Most of a Software Engineering Course

Fatma Meawad

German University in Cairo
fatma.meawad@guc.edu.eg

Abstract

Software engineers are expected to possess a variety of technical, social and personal competencies to be well prepared for real world working environments. At the German University in Cairo (GUC), we were able to guide large groups of students into becoming self managing and self learning communities whose members work together as a team to build large scale software. The students were able to experience many of the challenges in software engineering and develop a variety of related skills in a short period of time. This report describes our detailed experience in reaching such results using agile development practices in a simulated enterprise environment. With an aim to enable educators realise the same success, this report serves as a guide for educators who wish to replicate the process. The resulting successes and the concerns from this unique experience are discussed along with future recommendations.

1. Introduction

Spreading best practices in software engineering to industry is the ultimate goal of software engineering instructors [4]. However, in practice, the reverse is taking place. Employers complain from the unreadiness of university graduates for real world working environments and their unfamiliarity with related best practices such as, the use of version control, automated testing and collaboration tools. The lack of social and personal skills that will allow them work smoothly in a team is another big concern [3]. Such concerns exist even for ranked students with advanced programming and problem solving capabilities.

Researchers at Carnegie Mellon University have recently introduced a framework to help educators and employers understand the various roles a software engineer might take and their competencies [5]. On a higher level, the identified competencies fall under three main categories: technical, social and personal. The responsibility of equipping students with such competencies lies on the shoulders of academics, yet the context of a University creates several obstacles for software engineering instructors. In this report, we introduce a method for teaching software engineering practices using agile management and development techniques derived from SCRUM/XP methods. Additionally, in order to make the most out of the course and enrich the students' experience, we applied this method in a close simulation of an enterprise with multiple companies. Furthermore, in order to increase the students' motivation, a competition for the best company is announced at the beginning of the course. The proposed method was applied for two consecutive years with notable success that does not rely on students' individual technical skills.

We start this report by a background of the specific context of the course and the associated challenges. Then, we describe how SCRUM/XP was used to meet the software engineering learning

objectives while maintaining the feeling of a real world enterprise simulation. We detail the assessment and evaluation techniques used and the related tools in order to enable interested educators to smoothly integrate the discussed approach. We end the report with a discussion of the successes and concerns along with future improvements based on the lessons learned.

2. Background

The German University is a relatively new educational institutional which was established in 2003. Software engineering at the University is taught to third year Computer Science and Business Informatics students. The total number of students ranges from 130 to 170 students scheduled in groups of maximum 25 students per group resulting in 6 to 7 groups registered to the course. There is one lecturer and one teaching assistant who are assigned to the course.

2.1. Motivation

From the year 2005 till 2008, software engineering was taught either as a completely theoretical course or with a small project at the end of the semester. Senior students who undertook the course during this period had very little appreciation of the software development process. It was looked at as a trivial task that can be done individually without prior planning. Such beliefs were even more prominent with top ranked students who have excelled internationally in programming competitions. Almost none of the senior students was familiar with version control, automated testing or the challenge of changing requirements.

It was necessary to reconstruct the course to prepare the students for the various roles of software engineers and their basic competencies. Just following a traditional classroom based or project based course was not going to change the strong attitude of computer science students against software engineering. We had to take an additional step of taking the students out of the classroom environment. Thus, we organised a close simulation of a real world working environment where large teams work to design, implement and test software systems. The proposed software should be rich enough requiring multiple teams working together and posing complex interactions, coordination and collaboration challenges. We were also keen to make the students appreciate the value of the artifacts they are producing and not just do them for the sake of grades or obligation. For example, frequently updating the architecture document or the issue tracker is a must that forces itself in a team of 25 members who are trying to meet a strict deadline.

2.2. Using Agile Development Methods

Most current approaches in teaching software engineering courses focus either on unstable requirements and design or the rigorous development process [2]. In the meantime, agile processes aim to address the challenge of changeable requirements, rigorous development to frequent deadline under strict deadlines with a light weight changeable design. We believe agile processes bring the right mix of practices for teaching software engineering. Among the existing agile methods, we chose Scrum/XP for designing the course.

2.3. Why Scrum/XP ?

Scrum is one of the most popular agile management methods. It scored the highest percentages of adoption among other methods according to statistics reported by VersionOne ¹. Its success is

¹<http://www.versionone.com/>

associated with giving the ownership of the development process back to the team members [6]. It is usually used with extreme programming development practices as they are seen to complement each other. XP practices were promoted for Software engineering education in several publications [1, 8], with reported students' acceptance of its practices [7].

Scrum teams have to be cross-functional, self-managed, self-organized and self-empowered. Achieving such team dynamics requires strict adherence to three main practices of Scrum among the team members: high transparency, continuous inspection and adaptation. Such dynamics can be very appealing in a University setting for several reasons:

- Cross-functionality proposes equal value and various roles for each team member. In other words, the expected outcome from the course is assumed to be fair and rich for each student.
- Self managed and self organised teams turn the challenge of large number of students into a privilege since this workforce will be employed in favor of the team. Such sustainable community can take some effort off the teaching team as students become more and more independent. Consequently, the continuous increase in the number of students will not be a major concern anymore.
- The stress on high visibility and transparency will require students to document their work and progress in various ways throughout the process. This will help us avoid the pattern of develop first and submit whatever pleases the instructor at the end. Students will need to document since their peers' work will be dependent on their documentation.
- Frequent releases, sprints and early results allows constant exposure to results and students' activities. The first sprint is usually educational where students see the results of their work and overcome the overwhelming feeling of working in such a large team.

3. Design and Delivery of the Course

The virtual agile enterprise approach was applied for two consecutive years to teach software engineering at the GUC. The course has one lecturer who acts as a product owner and guides the scrum masters. At the beginning of the course, each group of students (25-26 maximum) is declared as a company with a randomly assigned name, for example, smartsoft, coolsoft, megasoft, collabsoft, etc. The companies are set to compete on developing a certain large scale software project with a predefined set of components. Every company is divided into random teams where each team is assigned to a component of the software under development. Every company is assigned to a scrum master by the lecturer. The scrum masters are junior teaching assistants who guide and coach the companies into following Scrum. All the course participants meet for a lab and a lecture every week. The rest of this section presents a brief outline of how the course activities.

3.1. Pre-Course preparation

The following are some important points that need to be finalised before starting the course:

- The teaching team should be ready with a software project idea. The idea should be rich with enough tasks for 25 team members who would take up to three tasks each. The first year of the course, we implemented a fully featured library system and in the second, we implemented an project management tool. It is important to be able to divide the presented project into at least five components to match the maximum number of components (teams)

present in one company. It is desirable, yet not necessary that the components are as loosely coupled as possible.

- The Scrum masters should be capable to technically guide Scrum teams; however, the emphasis when picking them should be on their patience, reliability and ability to deal with conflicts.
- The choice of the collaboration and development tools should be finalised and tested by the teaching team before the first lecture is given. As will be discussed later, we mostly rely on Google collaboration tools and open source development IDEs, such as Eclipse.
- The lecturer should secure a dedicated place for the upcoming Scrum activities. For example, companies will need to schedule scrum meetings for components, for the whole company or for scrum of scrums. In our case, there was only one lab available and a Google calendar was shared with all course participants to organise the room reservation.

3.2. The First Week

Scrum/XP in this context cannot work without strict adherence to its rules. Thus, in the first meeting the lecturer should stress the course rules after introducing the whole organisational structure of companies, managers, leads, scrum masters and product owner. The lecturer makes it clear to the students that they will be treated as employees and there is a high degree of independent learning and discovery. Throwing many unusual conditions and course rules at the students can be overwhelming. The first meeting is usually a challenging meeting where the lecturer should motivate the students, yet convey how serious the process is. Thus, it is recommended to showcase previous results if exists and discuss the importance of learning such practices for their career.

In the lab following the meeting, the companies are requested to elect a company manager and team leads. All interested students nominate themselves and the voting takes place during lab meetings. It is important to note that company managers and team leads are mainly representatives of their teams not figures of authority. They represent their teams when needed and submit attendance of meetings. There are not graded activities. However, the competition acknowledges best team lead and outstanding performance which is a good motivation for students to nominate themselves. In the lab meeting, they are also introduced to version control and the collaboration tools they are going to use. Every company had a Google folder for the collaborative editing of artifacts, such as the product backlog, sprint backlog, ERD, etc. For version control, Java.net educational program was the host of the companies' projects where a subversion repository was available for each company ². However, many students chose to use Google code project hosting. After a simple introduction and with some video tutorials, the members of every company are required to commit their profile in the company's profile to experience how 25 people edit one file.

3.3. User Stories

The first topic covered in the course is how to extract user requirements in the form of user stories and create a product backlog which is the XP approach towards collecting requirements. The students are given a broad description of the system that needs to be built. This resembles what could happen in real life where the description is not enough and there is much room for misunderstandings. The description has high level indication of components dependency and priority.

²<https://se603.dev.java.net/>

In Scrum, the product owner represents the customer and works as part of the team. The lecturer and the scrum master play the product owner role in the user stories meeting. Every company comes to the meeting prepared with their current understanding of the requirements and their questions. The lecturer and the scrum master discuss the requirements with the students and answer their questions. From the same description, different companies will come up with some different user stories which is a great example to the students about how requirements can be misinterpreted.

An excel sheet with a predefined template of a product backlog is created in the company's folder to be collaboratively edited by the team members. Every component is encouraged to work on the stories that relate to them together while editing the product backlog in their meetings. At the end of the third week, every company has a prioritised list of user stories, scenarios, related components driven from the original requirements description. The product backlog is checked by the scrum master and the lecturer for adherence to the template and for basic understanding of the concept. Being agile, the product backlog at this stage is expected to easily accommodate changes and it should be refined and updated continuously. Thus, it is not necessary to check for full clarity and completeness since it is not necessary at this stage.

3.4. A Simple Design

Junior students enter software engineering with a good background on database design. Every company is required to conduct a meeting with the presence of the scrum master to brainstorm on the entities required for their application and the relations between them. In the same meeting, the company should elect a database owner who is going to be the only one with edit permissions to the database schema. The database owner is expected to approve and announce to the whole company any changes to the ERD. Any tool can be used to create the database design collaboratively and manage the edit permissions, for example, Google drawing or SVN.

In the following week, the lecturer goes through what an architecture is and its different types. Every company is requested to prepare a simple description of their structural architecture. This requires that they finalise their framework choice since many frameworks impose a specific architectural pattern such as Model View Controller (MVC). Additionally, every company is already divided into components, therefore every company was required to hold a meeting where the product backlog is used to define the dependencies among the components. The dependencies are identified and newly identified stories are added to the product backlog. The architecture meeting is usually very useful in refining the product backlog and the priorities of the user stories.

3.5. Planning

The planning meeting is known to be a long intense meeting where team members and company members choose the high priority stories from the product backlog, break them down into tasks, estimate their effort. After the user stories and the design meetings, every company is ready to do sprint planning. The number of sprints is declared earlier by the lecturer according to the course timeline. Setting the sprint duration to two weeks was the best choice to fit the course timeline and allow from 2 to 3 sprints in the course. The lecturer covers how sprint planning takes place and demonstrates the effort estimation game with the scrum masters for the students. The students are also introduced to task boards and how they are used throughout the process. Finally, a sprint backlog template is shared with the companies to guide them to its structure and important values.

Another important activity that takes place during planning is the need to define when a task is considered complete or "Done". The definition of "Done" could differ according to the needs of

each course. In our course, a task is considered complete only when it is modeled (UML), implemented, documented, unit tested, acceptance tested and reviewed for all the previous activities. Team members are all required to sign up for the reviewing task since it is mandatory. After finalising all these items, they are added to an issue tracker where the assignee, reviewer, reporter, etc are filled in. The issue tracker helps tremendously in keeping track of the team's progress especially the reviewing process.

3.6. During the Sprint

The start of the sprint marks the start of the implementation phase where all team members are asked to work on their planned tasks. However, they are also required to have daily short scrum meetings to update each other. We allowed conducting such meetings online provided that they save the discussion logs. Attendance of meetings during the sprint is taken as serious as all the previous pre-sprint meetings. The scrum master takes various measures to monitor the attendance, either through unannounced show up to the meetings or logging to the scheduled online sessions. During the sprint, it is not allowed to change the product backlog. The sprint backlog and the issue tracker are being regularly updated. As implied earlier, a team member cannot mark a task complete or done without the approval of the reviewer.

3.7. Sprint Demo and Evaluation Techniques

At the end of each sprint, every company gets ready to show a demo of their work to the product owner. The product owner looks at the application as a whole without care to the interface. However, the scenarios in the product backlog should be there as specified in the product backlog. The product owner gives feedback and requests changes if any. The scrum master on the other hand, looks at the code and checks for documentation and unit tests. The scrum master runs unit testing tools or any other test suites. The peer review activity is taken very seriously that a reviewer is questioned for the quality of the assigned task.

The proposed course is full of activities and it keeps students, the lecture and the scrum masters busy throughout the semester. Continuous monitoring of punctuality, attendance, quality of the submissions and conflicts can be hectic. In many cases, collaboration tools can help the teaching team do the monitoring effectively and seamlessly. For example, the analysis of history and logs of code repositories gives good visualisation of the contribution of students over time. Such information helps in giving comments on the frequency of the commits making sure students do not just commit their work directly before the deadline. This is crucial since team members' work is dependent on each other. The same evaluation can be done through the Google docs which keeps simple history of modification which is good enough for the context of this course. This feature is crucial in frequency of updates of efforts on the sprint backlog. When it comes to grading, the students are graded throughout the process based on their weekly submissions; but the most important grades are the sprint grades. These grades are mostly based on the definition of "Done" as mentioned in section 3.5, for example, Quality (documentation, implementation, UML and reviewing), Proper use of Issue tracker and SVN, Sprint backlog update, meetings attended and all points submitted.

3.8. Sprint Review, then Repeat

In the sprint review activity, every company organises a meeting in which the members look back at their good and bad practices during the sprint. Every company is required to propose solutions that will address their weaknesses and emphasise the good practices during the following sprint.

Additionally, the product backlog is refined after the demo taking in consideration all the comments of the product owner.

The first sprint marks the serious independence of the company and its development into a community. After this, every company goes through the same steps again till the sprints are over and the final demo is scheduled. The first sprint is usually an educational sprint where the students become aware of all the aspects of the process. They would have done everything for the first time and they become ready for the following sprints.

4. Results

The method described in this report was applied for two years with 130 students in 2009 and 151 students in 2010 who come from computer science or business informatics background. At the beginning of the course, the students were shocked from the idea of working in a team of 5 members that is another part of a big team of 25 members. Many believed that it was impossible. Thus, there was the challenge of the teaching team to prove to the students that following software engineering practices can make this work. Over two years, twelve out of thirteen companies were able to deliver a working product with valued contribution from all the 25 members of each company. Half of the delivered products were of an advanced quality. Last year's winning company's product was continued through summer internships for real use at the University.

4.1. Students' Feedback

Even though, students were not asked explicitly to give feedback on the course, students were expressing their opinion at the end of the course through their companies' Google groups, personal blogs or microblogs (twitter). Analyzing 60 comments from students' feedback shows outstanding appreciation to the course benefits not only on a technical level but also on a personal and social level.

Fifty eight percent explicitly praised their experience in the course describing it as 'life changing' and "a turning point". The students described the course as the "best" course they took at the University and "a cocktail of fun-hard work". One company manager sent a message thanking the company and the teaching team saying "...We've learnt how to take responsibility, to perform well under pressure, to work [with conscious] on our tasks, to take challenges to the next level – always". What was even more interesting is that two students discussed how this course changed their attitude towards the University and got them to care about their studies stating that "... It even got me motivated to care more about school." and "...it really has motivated me to work and focus on all the other subjects too.". In the meantime, two students mentioned that they enjoyed the course even though the course load affected their other courses.

The company simulation helped develop a sense of ownership among the students where they became emotionally attached to their companies. In forty percent of the comments, the students refer to themselves with the company name, for example Smartsoftian, Coolsoftian, etc. Such feelings among the students were also clear during the course. The students made logos, shirts, advertisements, external website, Facebook groups, twitter accounts to promote their companies. The students became well known in the University and in other courses by their companies' names. Many company managers were creative in the way that they motivate the company members with, for example dish parties, candies or titles of recognition. All students comments were positive about their team members and their appreciation of their company managers.

The recommended practice of focal points extended beyond a single team. Many students were motivated to conduct sessions for all students about existing frameworks or tools that can be helpful

for them. All these activities were initiated by the students without any recommendations from the teaching team. There was a remarkable team and family spirit that spread among the students. Two students discussed in their comments how they believed that working individually is much better till they took this course. The comments also shows that long term social relations evolved among team members.

Another promoted practice that showed major success among the students is the peer reviewing. Pending the completeness of a task on peer reviewing of code, documentation and testing was taken very seriously that a reviewer could keep sending comments and changes for up to four times before approving the completeness of a task. This practice developed a seamless learning and teaching mechanism among peers for both the reviewer and the student under review.

4.2. Concerns and Problems Faced

In both years, the company with the top ranked students did the worst performance among all companies. In fact, in the second year, the top ranked students' company was not able to deliver even a basic working application. It happened that in both years, the top ranked students were opinionated and stubborn causing major problems in communication. They actually decided to divert from the path of the course and reorganise themselves in an architecture team, testing team, design and implementation. The positive side of this is that such companies gave a great example of what not to do for all other companies. They gave a real live example that excellent programming skills are not enough to build software. Seeing this in a company with more than 80% excellent programmers was the best learning example and more effective than a whole lecture on the topic.

Some of the success factors mentioned caused some points of concerns as well. For example, some students became obsessed with winning the competition causing them to request adding features at the last minute, to work with a faster pace, or put pressure on peers to work more since they have to win as a team. These were rare cases, yet such focus on winning the competition was shifting the focus of the whole process. Some lecturers of other courses actually complained that the students did not want to work except on this course which caused them problems. Our approach towards dealing with such concern was announcing that any extra features added will not be used when judging the winning company. Additionally, if an extra feature is added without following the same process as all user stories, then it will be counted negatively in the competition.

5. Discussion

Preparing university students to the various roles of software engineers and their basic competencies is the ultimate aim of software engineering instructors. Achieving this ultimate goal is a complex task. In this paper, we described a new method in teaching software engineering that makes use of agile development methods, specifically Scrum/XP. Agile development methods can be great candidates for teaching software engineering due to the rate of visibility, flexibility, adaptability and traceability. The course was organised to put the students in a virtual enterprise environment where each group of students work and act as if they are recruited in a real company. Additionally, in order to increase the motivation of the students, a competition was announced for the companies to compete in producing the best product.

In both years, the course ended with great results that were visible in (a) the application built by the teams, (b) learning and teaching each other advanced development tools in software engineering and (c) appreciating the challenges of software engineering and (d) developing team work and collaborative environment. Besides the benefits of using Scrum/XP, we believe that the idea of a virtual enterprise and the sense of ownership worked in the favour of these successes. Additionally,

the large of number of students turned from a disadvantage to an advantage by enabling extensive peer reviewing. It also enabled us to work on a rich software application that helped demonstrate most of the objectives of the course. Finally, the frequent releases and the ability to see their creation early in the process added to the motivation and the confidence of the students.

6. An Outlook

After witnessing the positive impact of simulating a real world working environment on changing the students' attitude towards software engineering, I envisage great success in formally implementing an authentic enterprise inside the university where students work to achieve the learning objectives of some courses. This should take place without actually attending classes, taking exams or any such forms of assessments. Such vision resembles the idea of having technical semesters where every student attends on daily basis and work on real world problems. The idea of a technical semester can be applied to other courses not just software engineering where the learning objectives can be achieved by completing tasks related to the development of the company. For example, the company can hire software engineers, usability experts, network administrators, etc. Finally, any basic concepts or theories can still be covered but delivered through technical talks or seminars not in a class room oriented manner.

7. Acknowledgment

There are many people who contributed to the success of the course, most importantly, Ahmed Abousafy, Saher El neklawey, Rama Wafaa and all the company managers. Additionally, the course would not have been possible without the support of the faculty's Dean, Prof. Slim Abdennader who facilitated all the necessary changes required for the course.

References

- [1] J. Bergin, J. Caristi, Y. Dubinsky, O. Hazzan, and L. Williams. Teaching software development methods: the case of extreme programming. *SIGCSE Bull.*, 36:448–449, March 2004.
- [2] S. Jarzabek and P.-K. Eng. Teaching an advanced design, team-oriented software project course. In *Proceedings of the 18th Conference on Software Engineering Education & Training*, pages 223–230, Washington, DC, USA, 2005. IEEE Computer Society.
- [3] A. LaSalle. An inverted computing curriculum: preparing graduates to build quality systems. In *Teaching and Learning in an Era of Change*, number 1, pages 280–284, 1997.
- [4] S. L. Mansar. Information systems application development courses: A carnegie mellon university experience in global pedagogy. In *IEEE Conference on Software Engineering Education and Training*, pages 129–135, 2010.
- [5] J. G. Rivera-Ibarra, J. Rodriguez-Jacobo, and M. A. Serrano-Vargas. Competency framework for software engineers. *Software Engineering Education and Training, Conference on*, 0:33–40, 2010.
- [6] K. Schwaber and M. Beedle. Agile Software Development with Scrum. *Series in Agile Software Development*, October 2001.
- [7] A. Shukla and L. Williams. Adapting extreme programming for a core software engineering course. *Software Engineering Education and Training, 2002. (CSEE&T 2002). Proceedings. 15th Conference on*, pages 184–191, 2002.
- [8] L. Williams and R. Upchurch. Extreme programming for software engineering education? *Frontiers in Education, Annual*, 1:T2D–12–17vol.1, 2001.